HouseVerdi Arithmancy

GenAI projekt och dokumentation för APL

Claudio Verdi | 24 oktober 2026

Jag har utvecklat och driftsatt HouseVerdi Arithmancy i AWS med stöd av AI-verktyg. Applikationen kombinerar programmerade numerologiska beräkningar med genererade texter från Amazon Bedrock. Den 24 oktober verifierade jag webbgränssnittet, serverns tjänster, ett direkt AI-anrop och API-validering. Koden och installationsinstruktionerna finns i mitt publika GitHub-kodförråd.

Syfte och användning

Projektet är en webbapplikation för personliga profiler och symboliska jämförelser mellan två personer. Användaren anger namn, födelsedatum och ett profilval för kön. Resultatet omfattar Pythagorean, Chaldean, Chinese Kua, Indian/Vedic samt färg och stjärntecken. Tre rollkort ger reflektioner för partner, vän och kollega.

Numerologi och kompatibilitetsprocent är underhållning och reflektion. De är inte vetenskapliga prognoser. AI-text kan innehålla fel, vilket också upptäcktes i ett av testerna och redovisas i rapporten.

Länkar och underlag

Webbapp vid kontrollen: http://16.16.184.122/
Kodförråd: https://github.com/HouseVerdi/HouseVerdi_Arithmancy

Rapporten bygger på mina skärmbilder och terminalutdata från den 24 oktober, projektfilerna och den tillhandahållna Antigravity-konversationen. Skärmbilderna dokumenterar den befintliga lösningen. De visar inte en ny fullständig driftsättning från en tom server.

Projektets start i Antigravity

Jag började i Google Antigravity, där projektets grund tog form. Den första idén var att jämföra två personers namn för partner, vän och kollega. Konversationen beskriver bland annat numerology_system.py och ett HTML-gränssnitt. Jag utvecklade kraven med födelsedatum, personligt läge och fler numerologiska system.

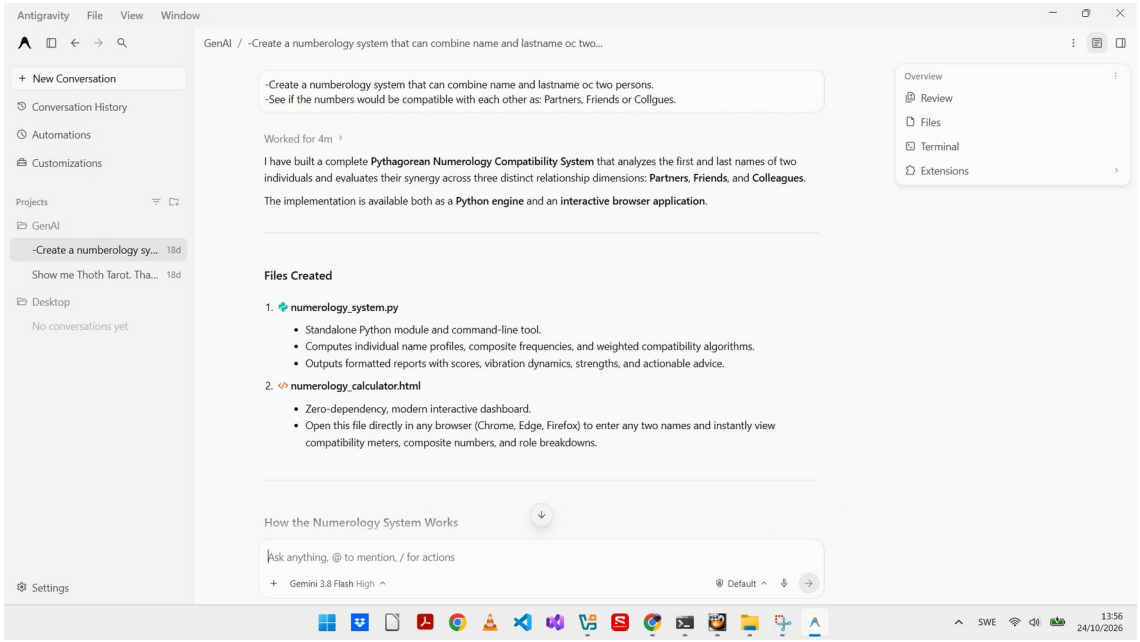


Bild 1. Den ursprungliga projektidén i Antigravity

När användningskvoten tog slut fortsatte jag med ChatGPT. Bilden visar meddelandet Individual quota reached den 6 oktober. Beskrivningar i chatten är historik; de senare testerna visar vilka funktioner som faktiskt fungerade.

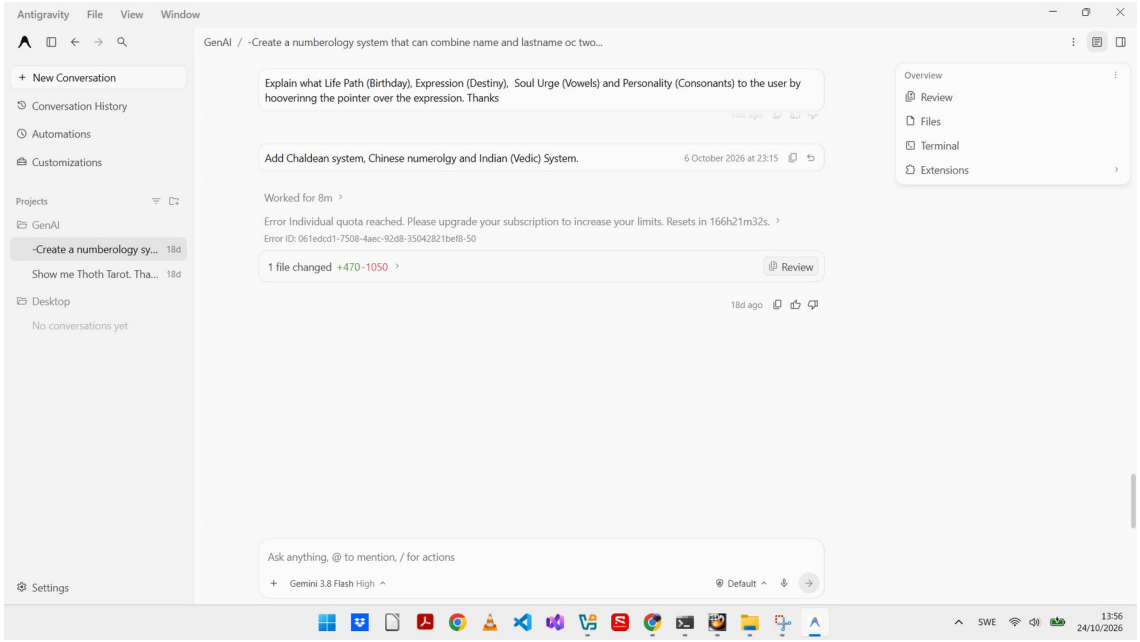


Bild 2. Kvotgränsen som ledde till byte av verktyg

Eget arbete och projektfiler

Jag styrde kraven och gränssnittets utformning, provade resultaten och genomförde kommandona för AWS, felsökning och GitHub. AI-verktygen hjälpte med kod och instruktioner. Efter Antigravity fortsatte jag bearbeta layout, personligt standardläge, Non-Binary som standardval, färger och AI-texter i de tre rollkorten.

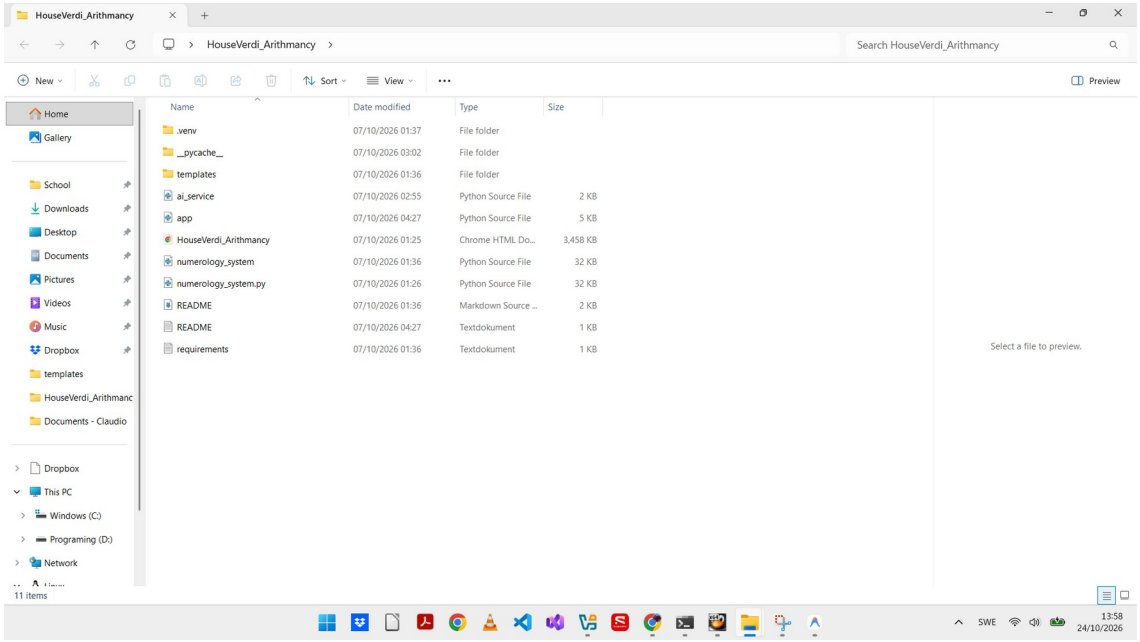


Bild 3. Den lokala projektmappen i Windows

Filer i det publicerade kodförrådet

app.py hanterar Flask-rutter, validering och AI-läsningar. ai_service.py anropar Bedrock. numerology_system.py innehåller Python-beräkningar. templates/index.html innehåller gränssnitt, JavaScript-beräkningar och API-anrop. requirements.txt anger beroenden. README.md beskriver projekt och installation.

Den lokala mappen innehöll även virtuell miljö, cache och äldre filer. Det publicerade kodförrådet använder utvalda projektfiler och .gitignore. Miljön .venv och Python-cache ingår inte i den första committen.

Systemets uppbyggnad

Webbläsaren hämtar sidan från Nginx över HTTP på port 80. Nginx skickar begäran vidare till Gunicorn på 127.0.0.1:8000. Gunicorn kör Flask. Vid POST /api/reading validerar Flask indata, räknar fram symboler i Python och anropar Amazon Bedrock med boto3. Svaret innehåller genererade texter för partner, vän och kollega.

Beräkning och genererad text

Gränssnittet gör egna JavaScript-beräkningar för de synliga talen och poängen. Backend gör även beräkningar i Python. Webbläsaren använder API-svarets interpretations för AI-texterna. Därför är det viktigt att skilja gränssnittets beräknade resultat från modellens fria formuleringar.

I den granskade koden tas namn, fullständigt födelsedatum, kön och polaritet bort från personobjektet innan symbolerna skickas till modellen. Flask returnerar fortfarande profiluppgifter till användarens webbläsare. Modellen uppmanas att svara med JSON och tre textfält. Programmet kontrollerar att dessa finns och visar AI-text med textContent.

Konfiguration och felhantering

```
BEDROCK_REGION=eu-north-1
BEDROCK_MODEL_ID=eu.amazon.nova-micro-v1:0
GET /
POST /api/reading
```

ai_service.py använder boto3:s credential-kedja och innehåller inga hårdkodade AWS-nycklar. EC2-bilderna visar en IAM-roll kopplad till instansen. Rollens fullständiga policy och credential-källa under körning granskades inte separat i denna kontroll.

Ogiltiga indata ger HTTP 400. Fel vid generering ger ett generellt HTTP 502-svar i den granskade koden. Gränssnittet avbryter gamla förfrågningar när formuläret ändras. HTTP 400 testades direkt; HTTP 502 och samtidighet belastningstestades inte.

AWS instans och operativsystem

Instansen HouseVerdi-Arithmancy har ID i-00f140425a029a738 och publik IPv4 16.16.184.122. Den körs i Stockholm, eu-north-1, zon eu-north-1a. Konsolen visade Running och 3 av 3 godkända statuskontroller.

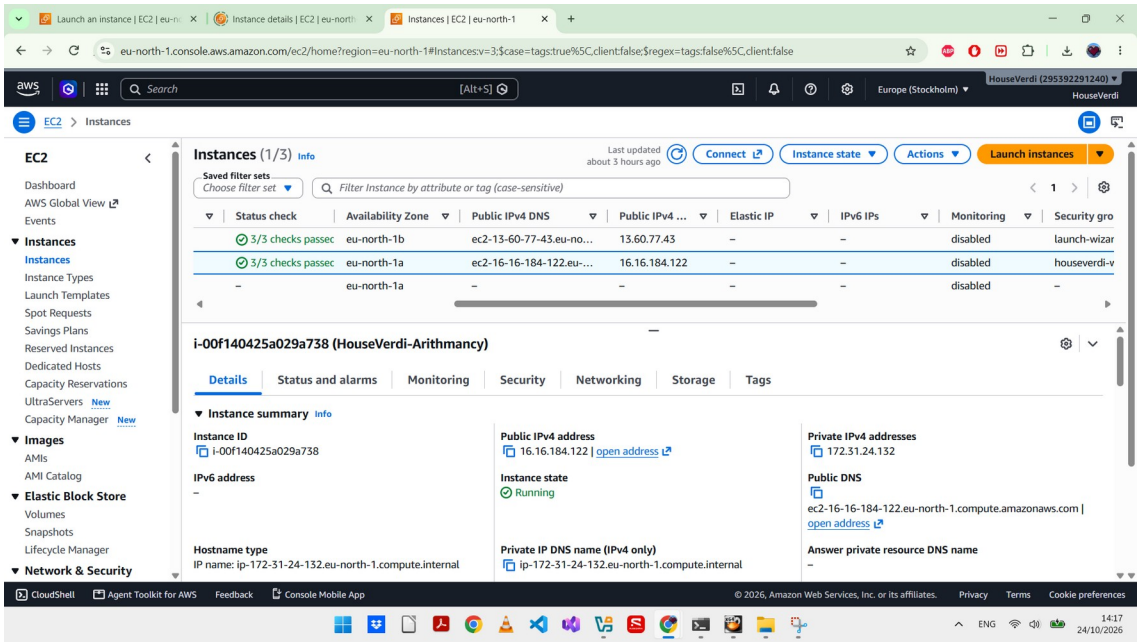


Bild 4. Den valda GenAI-instansen och nätverksadresserna

Instanstypen är t3.micro och den privata adressen är 172.31.24.132. IMDSv2 är Required. AMI-namnet anger Ubuntu Noble 24.04 amd64. SSH-inloggningen den 24 oktober visade Ubuntu 24.04.5 LTS. Instansens starttid i konsolen var 7 oktober 2026 klockan 04.09 svensk sommartid.

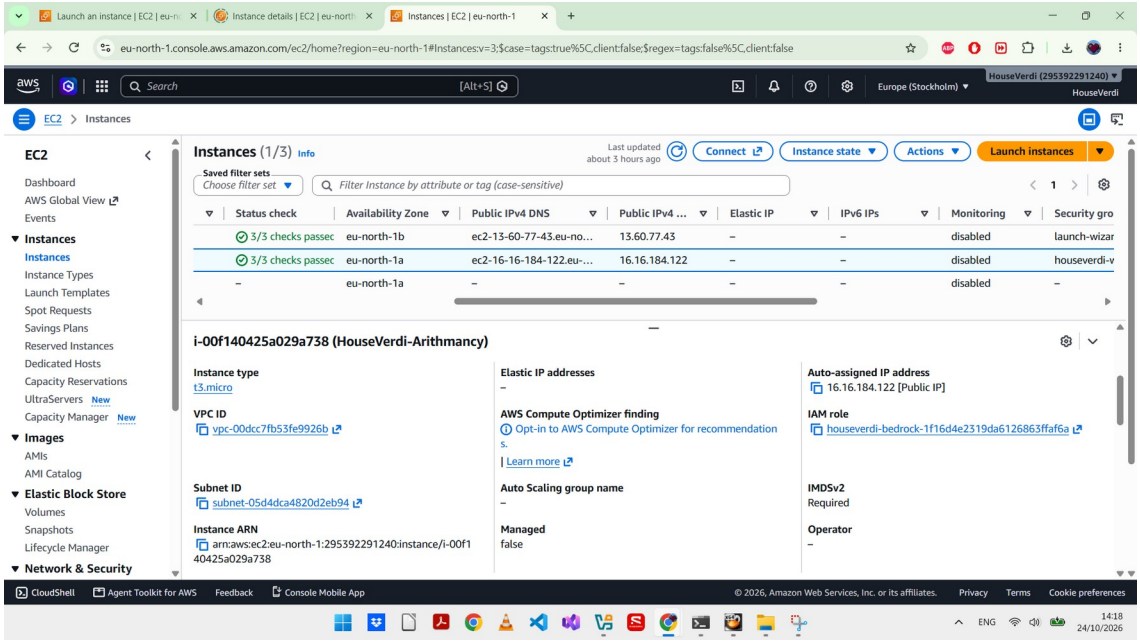


Bild 5. Instanstyp och ansluten IAM-roll

Nätverksåtkomst och SSH

Säkerhetsgruppen tillåter TCP 22 endast från 83.188.29.94/32 och TCP 80 från 0.0.0.0/0. Ingen regel för 443 visas i underlaget. Appen öppnades med uttrycklig http-adress efter ett tidigare timeoutförsök i webbläsaren.

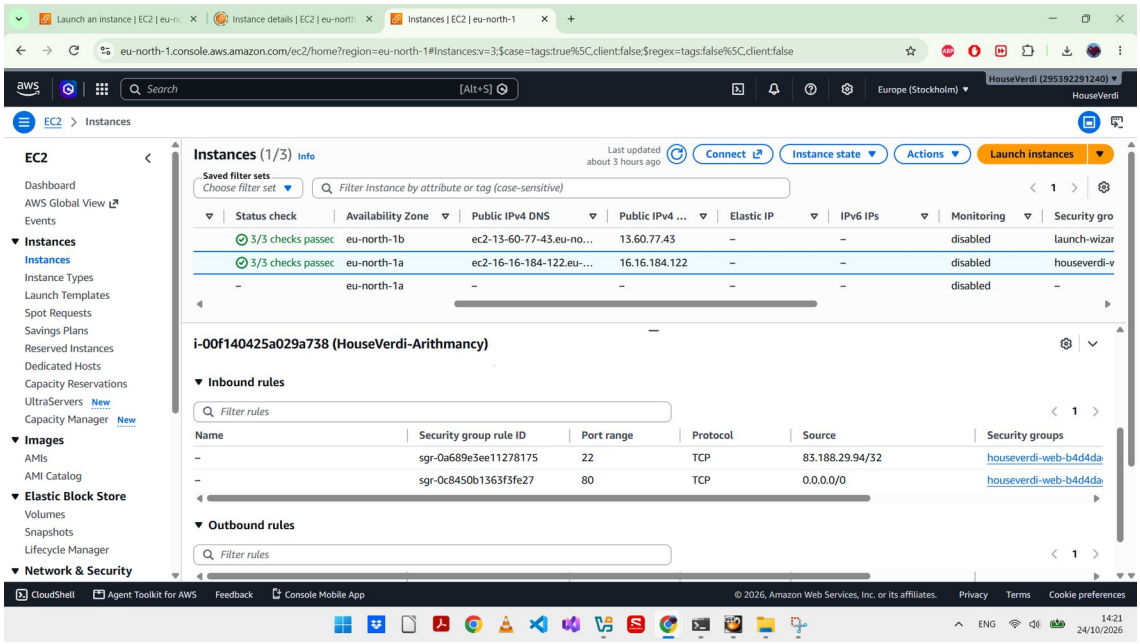


Bild 6. Inkommande regler för SSH och HTTP

En sökning efter PEM-filer hittade inte nyckeln. I .ssh-mappen hittade jag houseverdi_aws utan filändelse. Jag anslöt som ubuntu med denna nyckel. Skärmbilden visar filnamnet och lyckad anslutning, inte nyckelns innehåll.

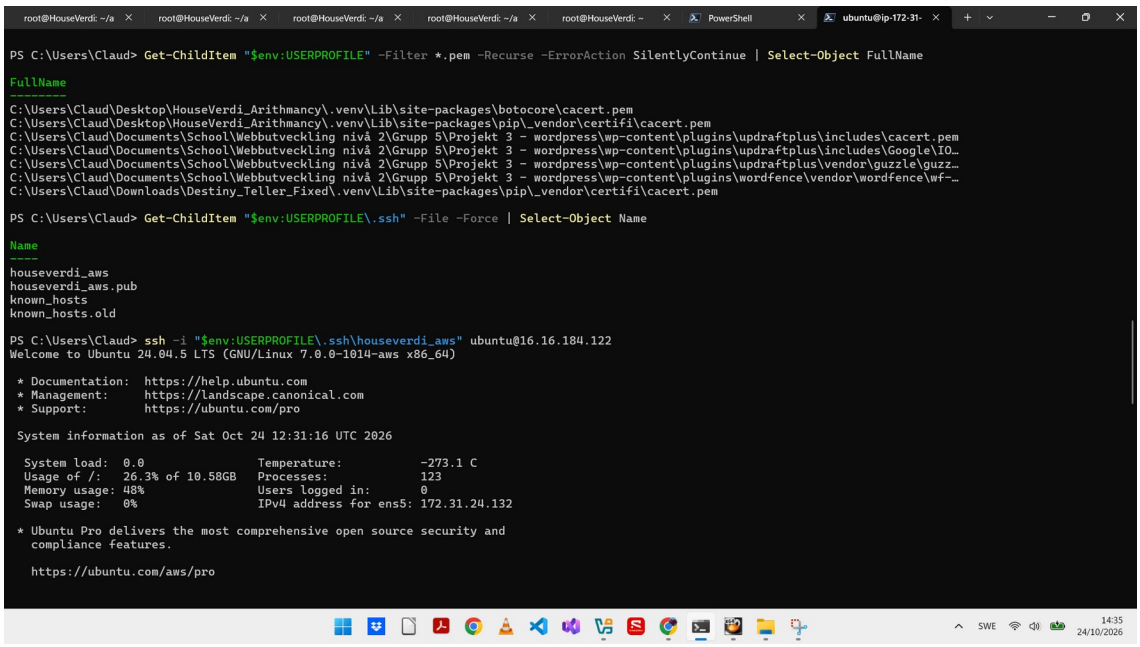


Bild 7. Anslutning från PowerShell med den befintliga SSH-nyckeln

Tjänster och reverse proxy

systemctl status visade att houseverdi och nginx var active running och enabled. Gunicorn 23.0.0 kördes med två workers och timeout 120 sekunder. Loggen visar bindning till 127.0.0.1:8000.

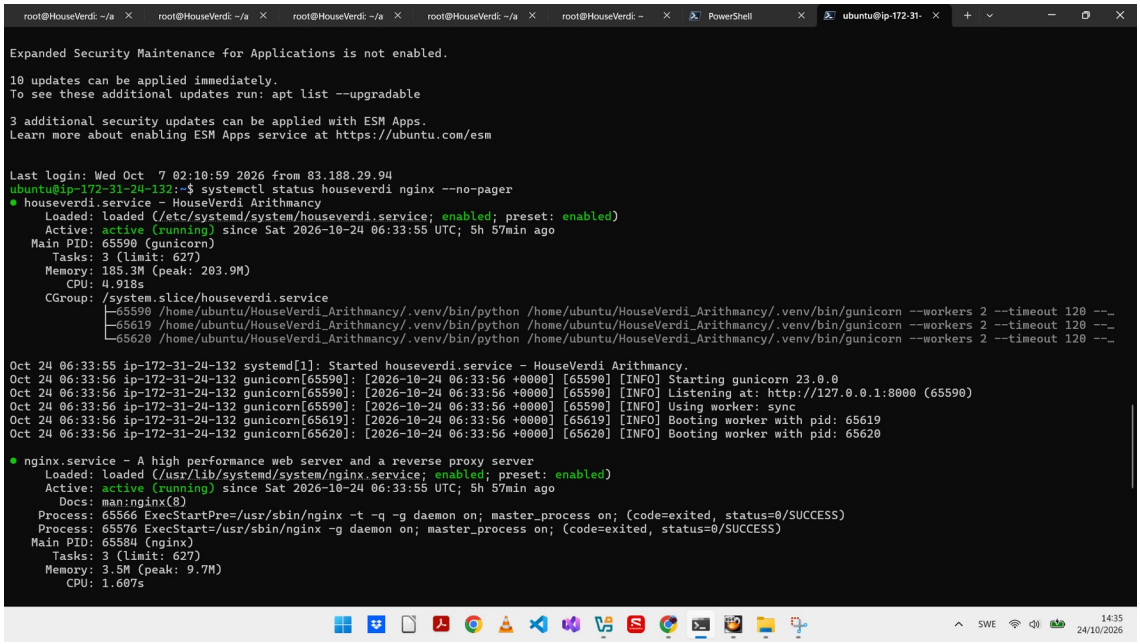


Bild 8. Aktiva systemd-tjänster och Gunicorn-processer

curl -I mot 127.0.0.1 gav HTTP 200 OK från nginx/1.24.0. nginx -T rapporterade godkänd konfiguration och proxy_pass till appen. Samma terminalkontroll visade rätt Bedrock-region och modell samt ett lyckat direkt AI-anrop.

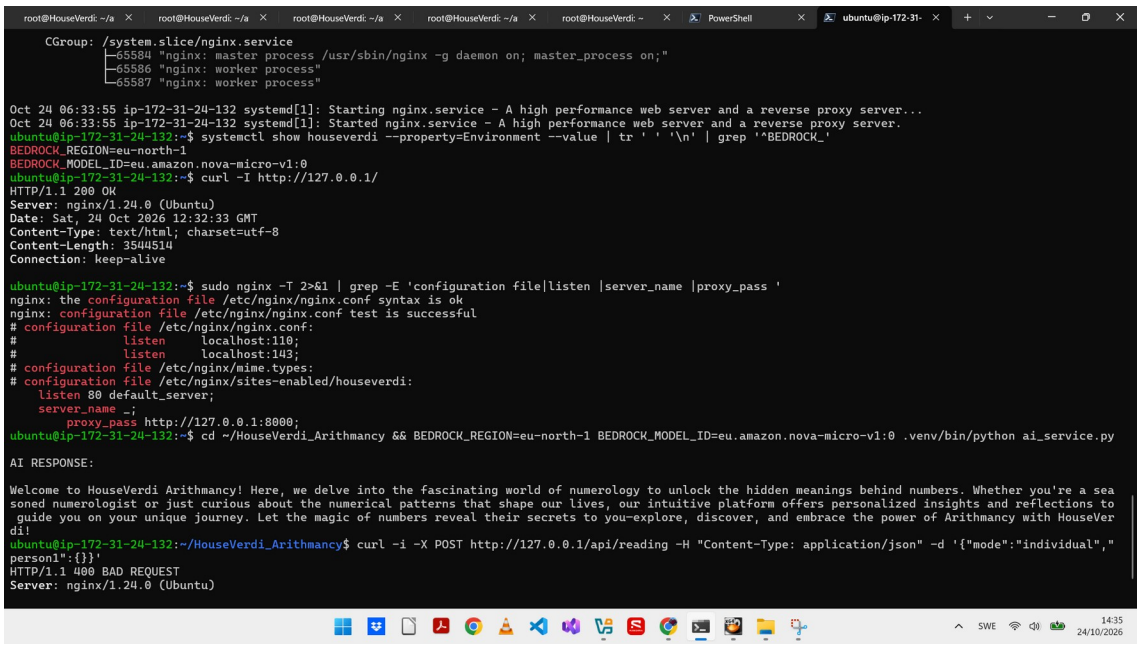


Bild 9. Bedrock-konfiguration samt HTTP och Nginx-kontroller

Test av personlig profil

Jag använde testprofilen John Doe, Male, född 15 maj 1988. Gränssnittet visade Taurus, Life Path 1, Expression 8, Soul Urge 8 och Personality 9. Färgresultatet var Ruby red med talet 1.

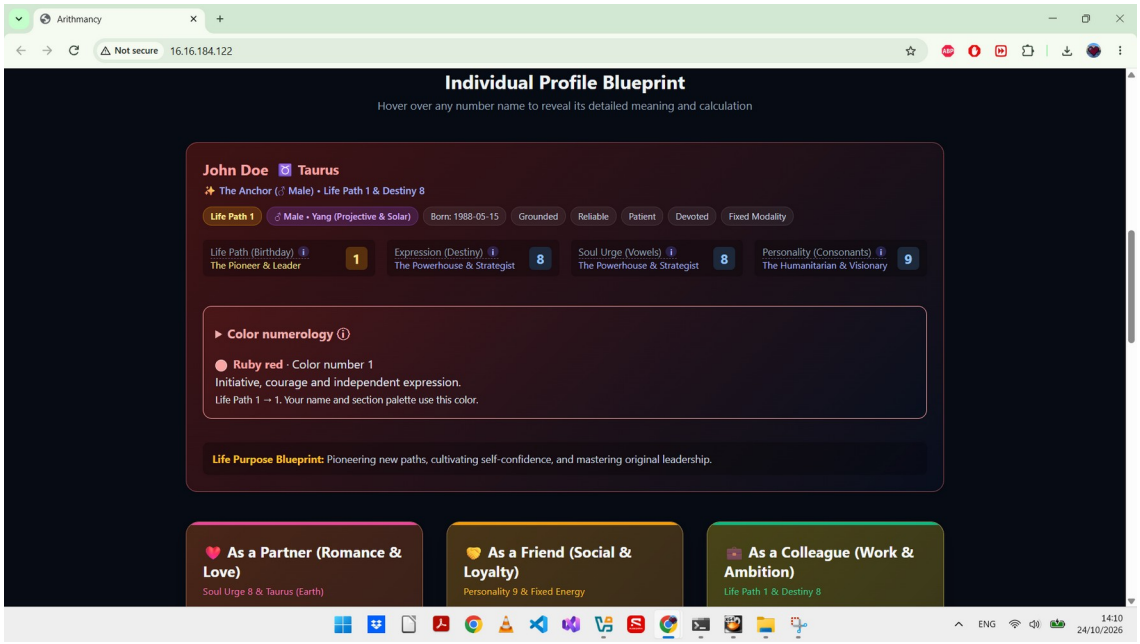


Bild 10. Personprofilens beräknade resultat

Alla tre rollkort visade AI generated guidance. Ett innehållsfel upptäcktes i partnertexten: den hänvisade till Soul Urge och Personality som 9, trots att Soul Urge var 8. Genereringen fungerade, men textens numeriska korrekthet var därmed inte fullständig.

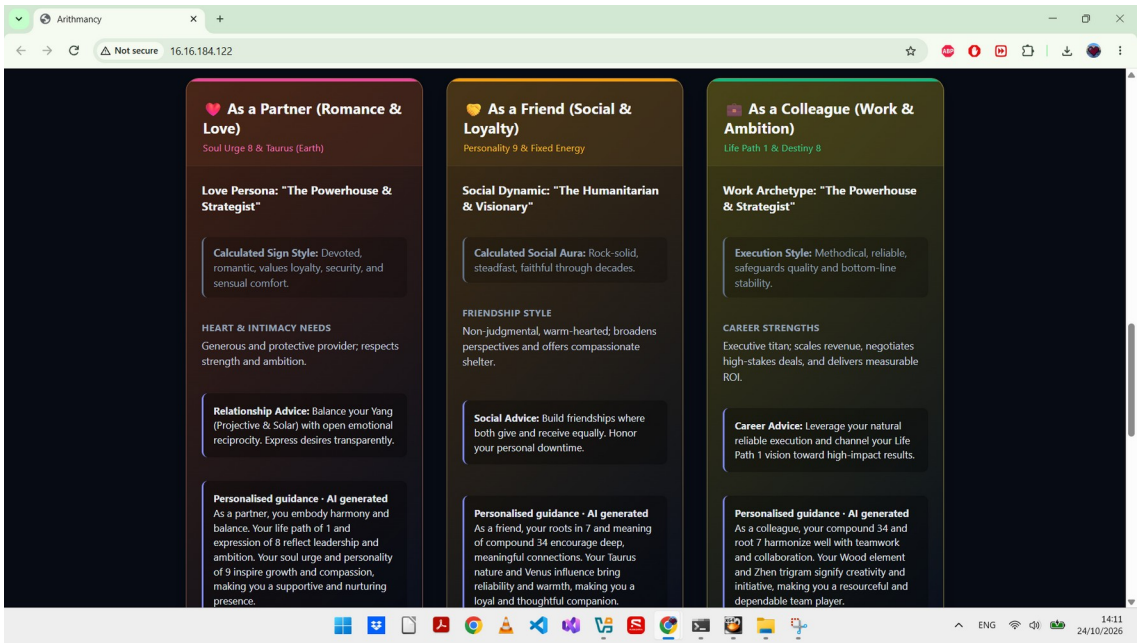


Bild 11. AI-texter för personlig profil med det upptäckta talfelet

Test av två profiler

Jag jämförde John Doe, Male, 15 maj 1988 med Jane Dame, Female, 25 april 1990. Janes profil visade Life Path 3, Expression 8, Soul Urge 3 och Personality 5. Detta är demonstrationsdata.

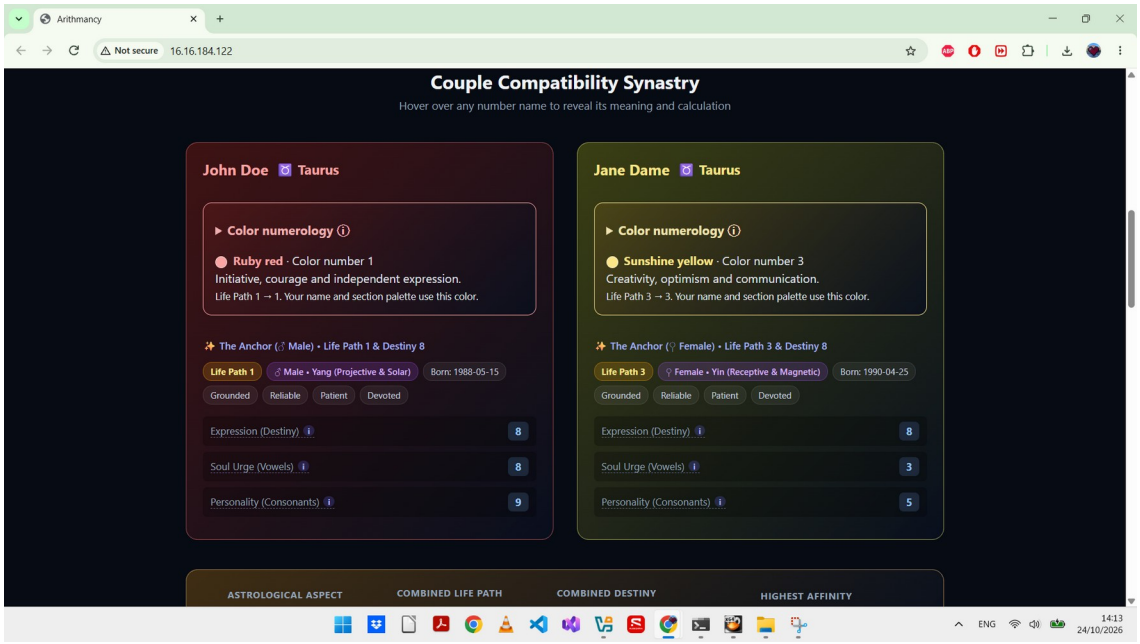


Bild 12. Indata och profiler i Couple Compatibility

Gränssnittets rollpoäng blev partner 87 procent, vän 90 procent och kollega 92 procent. Combined Life Path visades som 4 och Combined Destiny som 7. Procenttalen kommer från applikationens symboliska formler och anger ingen uppmätt sannolikhet.

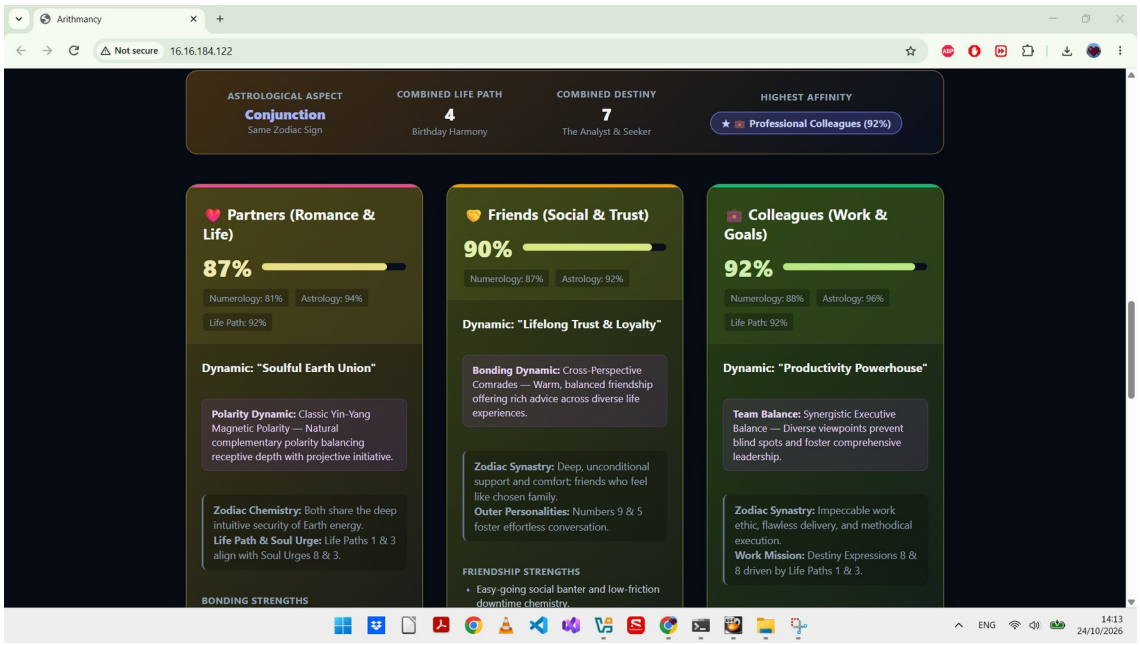


Bild 13. Rollpoäng och kombinerade tal

Resultat och AI för två personer

Systemjämförelserna visade Chaldean 90, Chinese Kua 74 och Indian/Vedic 92. Dessa är egna beräkningsresultat. Kua använder en approximation baserad på kalenderår och är inte ett fullständigt BaZi-diagram.

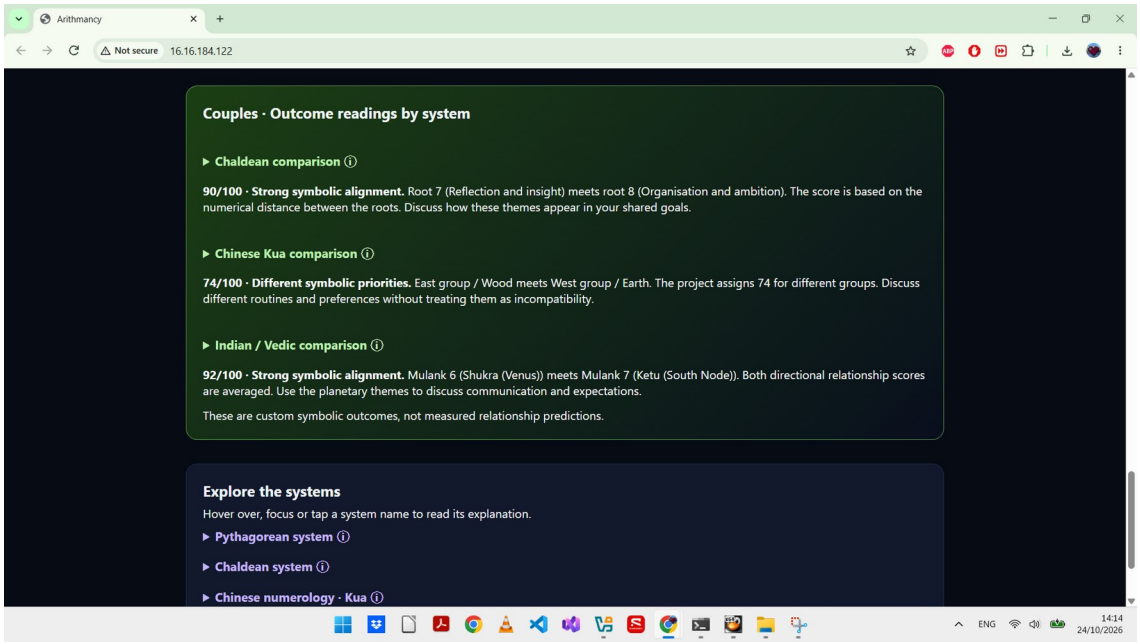


Bild 14. Systemresultat i jämförelseläget

Efter att AI-anropet slutförts visades genererade reflektioner i samtliga tre rollkort även i parläget. Det dokumenterar att gränssnittets API-flöde gav text i båda lägena. Ingen separat vetenskaplig validering av reflektionerna gjordes.

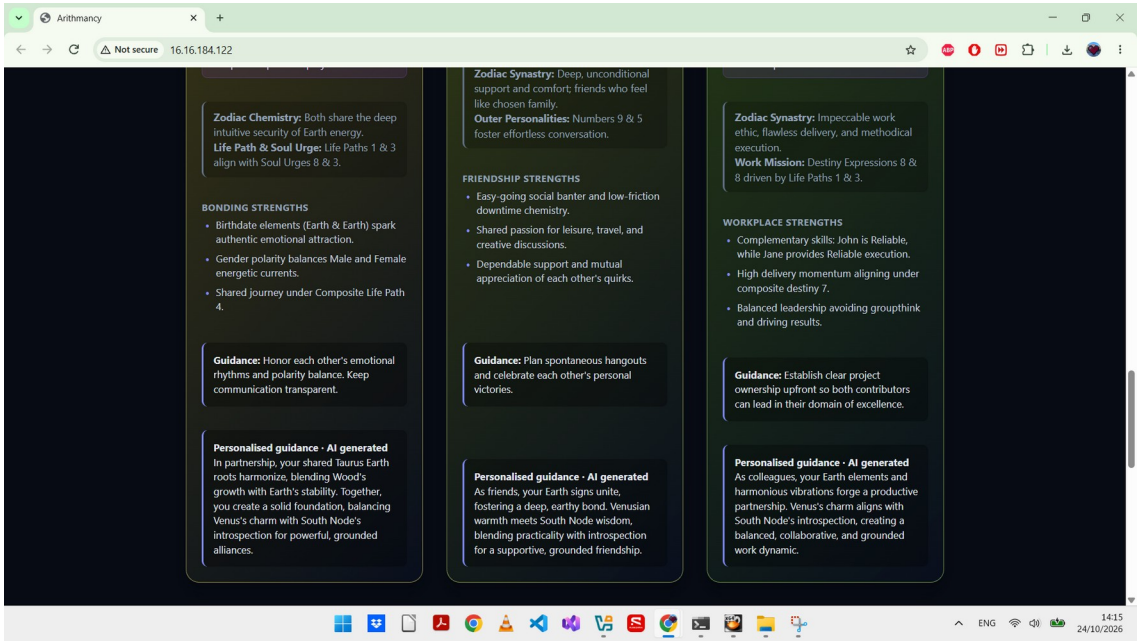


Bild 15. AI-genererade texter för partner vän och kollega

Direkt AI-anrop och API-validering

Jag körde ai_service.py i serverns virtuella miljö med BEDROCK_REGION och BEDROCK_MODEL_ID angivna. Terminalen visade AI RESPONSE och en genererad välkomsttext. Det visar en fungerande integration från EC2 vid testtillfället.

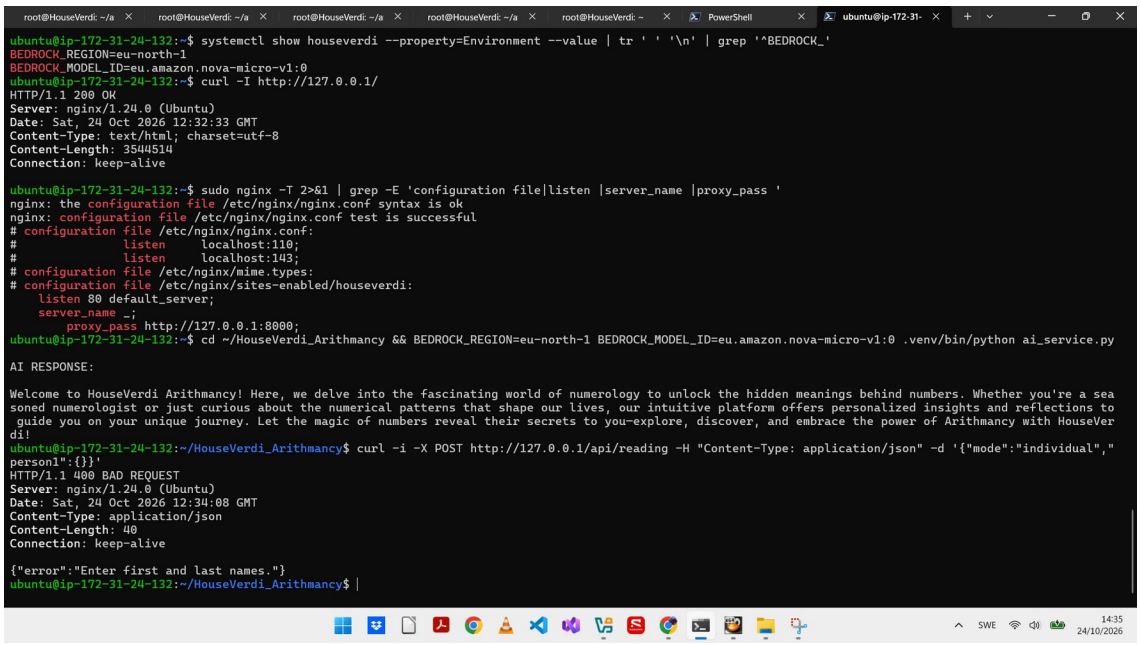


Bild 16. Lyckat AI-anrop och avvisad ofullständig API-begäran

Därefter skickade jag en JSON-begäran med mode individual och ett tomt person1-objekt till /api/reading. Svaret blev HTTP 400 BAD REQUEST med meddelandet Enter first and last names. Testet bekräftar att namnvalideringen avvisar denna ofullständiga begäran.

```
curl -i -X POST http://127.0.0.1/api/reading \
  -H "Content-Type: application/json" \
  -d '{"mode":"individual","person1":{}}'
```

Källkoden innehåller också kontroller av datum, framtida födelsedatum, profilval och AI-svarets tre textfält. Dessa kontroller har granskats i kod men utgör inte ytterligare direkt utförda tester i denna rapport. Ingen fullständig automatiserad testsvit eller belastningstest redovisas.

GitHub och versionshantering

Jag skapade ett publikt kodförråd under mitt personliga konto HouseVerdi. Projektmappen på EC2 saknade först .git. Jag lade till .gitignore, initierade grenen main och valde sex projektfiler. Git-identiteten sattes till Claudio Verdi med kontots noreply-adress.

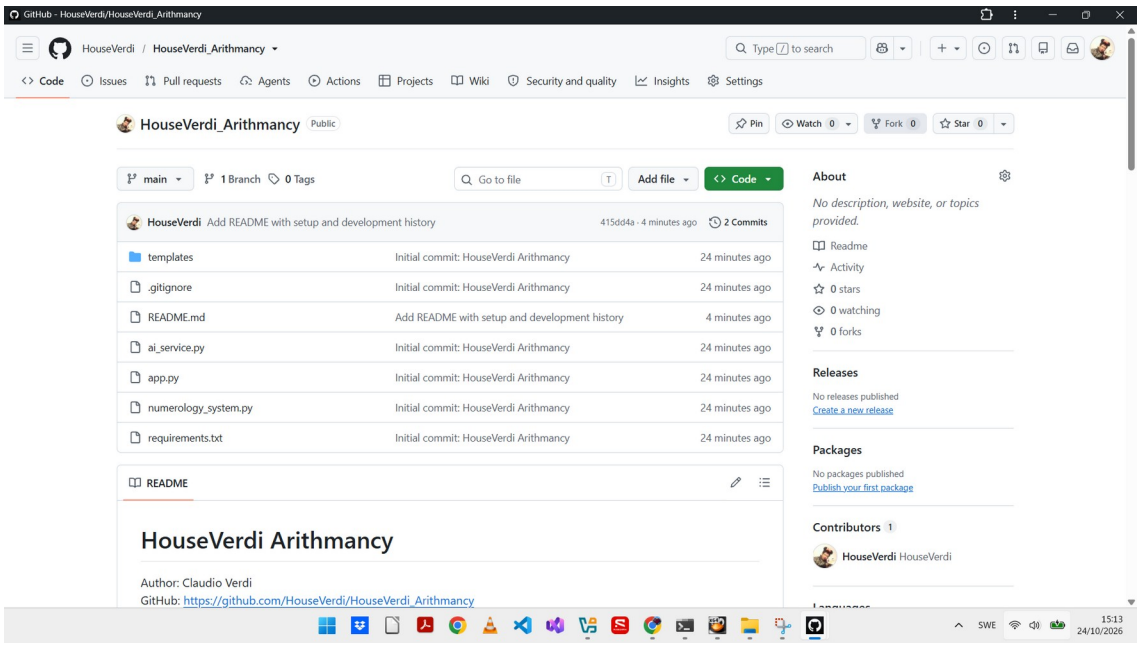


Bild 17. Publikt kodförråd med två commits och README

Första committen 3374b75 innehåller koden. Commit 415dd4a lägger till README med installation, historik, verifiering och begränsningar. Jag loggade in med GitHub CLI och laddade upp med git push. Inga inloggningskoder eller tokens återges här.

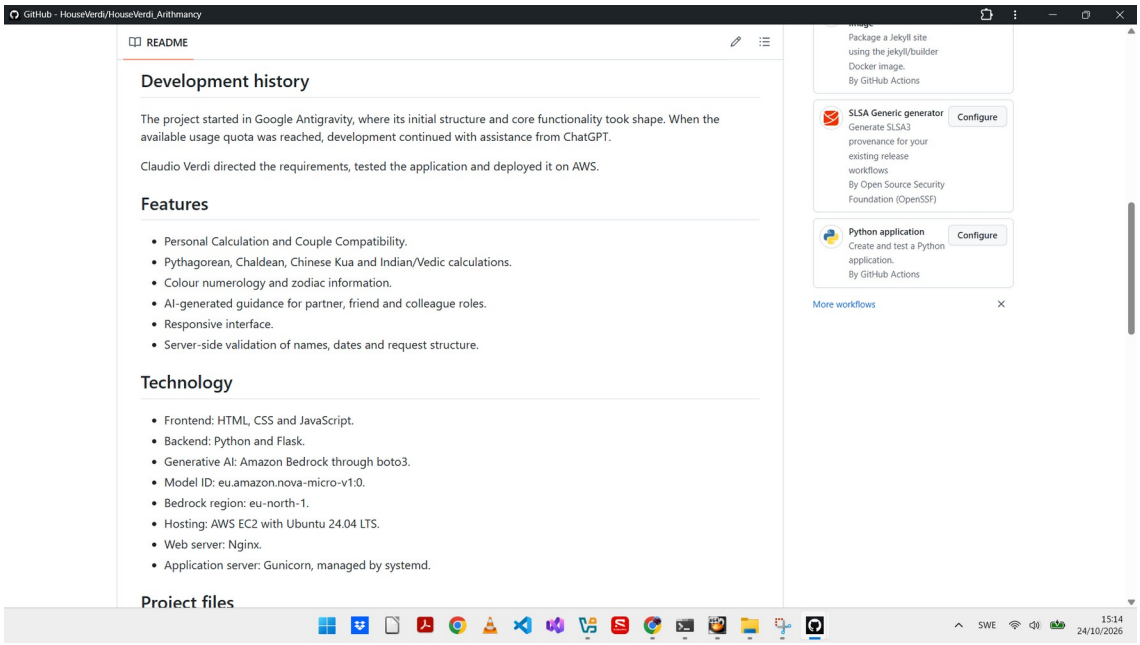


Bild 18. Publicerad utvecklingshistorik och teknikbeskrivning

Installation och drift

README beskriver lokal installation på Linux med Python, venv och Git. Beroenden installeras från requirements.txt. AWS-autentisering konfigureras separat för boto3 och måste ge behörighet att anropa vald Bedrock-modell. Nycklar ska inte sparas i kodförrådet.

```
git clone https://github.com/HouseVerdi/HouseVerdi_Arithmancy.git
cd HouseVerdi_Arithmancy
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export BEDROCK_REGION=eu-north-1
export BEDROCK_MODEL_ID=eu.amazon.nova-micro-v1:0
python app.py
```

Den lokala utvecklingsservern öppnas på http://127.0.0.1:5000. Den befintliga EC2-driftsättningen använder i stället Gunicorn på port 8000 bakom Nginx. Att öppna HTML-filen direkt från disken är inte samma sak som att köra Flask och dess API.

Kommandon för verifierad drift

```
systemctl status houseverdi nginx --no-pager
curl -I http://127.0.0.1/
sudo nginx -T
systemctl show houseverdi --property=Environment --value
```

Kommandona är referens för den redan installerade tjänsten. Kodförrådet innehåller inte en komplett automatiserad provisionering av server, IAM-roll, Nginx och systemd. En fullständig reproducerbar driftsättningsbeskrivning är ett fortsatt förbättringsområde.

Utvärdering och videoredovisning

Projektet visar en fungerande GenAI-funktion i en AWS-hostad webbapp. Jag kan följa flödet från formulär via Flask till Bedrock och tillbaka till gränssnittet. Jag har också publicerat koden med en README som förklarar hur projektet började och hur det körs.

Begränsningar och förbättringar

Det observerade Soul Urge-felet visar att instruktionen till modellen inte räcker för att säkra numeriska påståenden. Jag bör kontrollera modellens tal mot beräkningarna eller undvika att modellen återger tal. Jag bör även samla JavaScript- och Python-beräkningar i en gemensam källa, eftersom separata implementationer kan ge olika poäng.

Den nuvarande Kua-beräkningen använder kalenderår utan Li Chun-justering. Non-Binary följer den kvinnliga Kua-formeln i implementationen. Detta är en förenkling som bör vara tydlig för användaren. HTTPS, autentisering och begränsning av anropsfrekvens återstår som förbättringar. AWS-anrop medför kostnader och appen saknar här verifierad kostnadsuppföljning.

Videoredovisning

Jag har uppgett att GenAI_Claudio_Verdi.mp4 är inspelad och att jag kontrollerat ljud och bild. Inspelningen följde en genomgång av personläge, parläge, tjänstestatus, modellkonfiguration, direkt AI-anrop, API-validering och GitHub. Videofilen har inte granskats som bilaga till denna rapport, och någon oberoende kontroll av längd eller fullständighet redovisas därför inte.

Källor och underlag

Underlaget består av den tillhandahållna GenAI-projektbeskrivningen, Antigravity-konversationen och skärmbilderna, projektets Python- och HTML-filer, requirements.txt, publicerad README samt mina terminalutdata och skärmbilder den 24 oktober 2026. Tidigare README beskrev en äldre projektfas och ersattes i GitHub med den uppdaterade versionen.